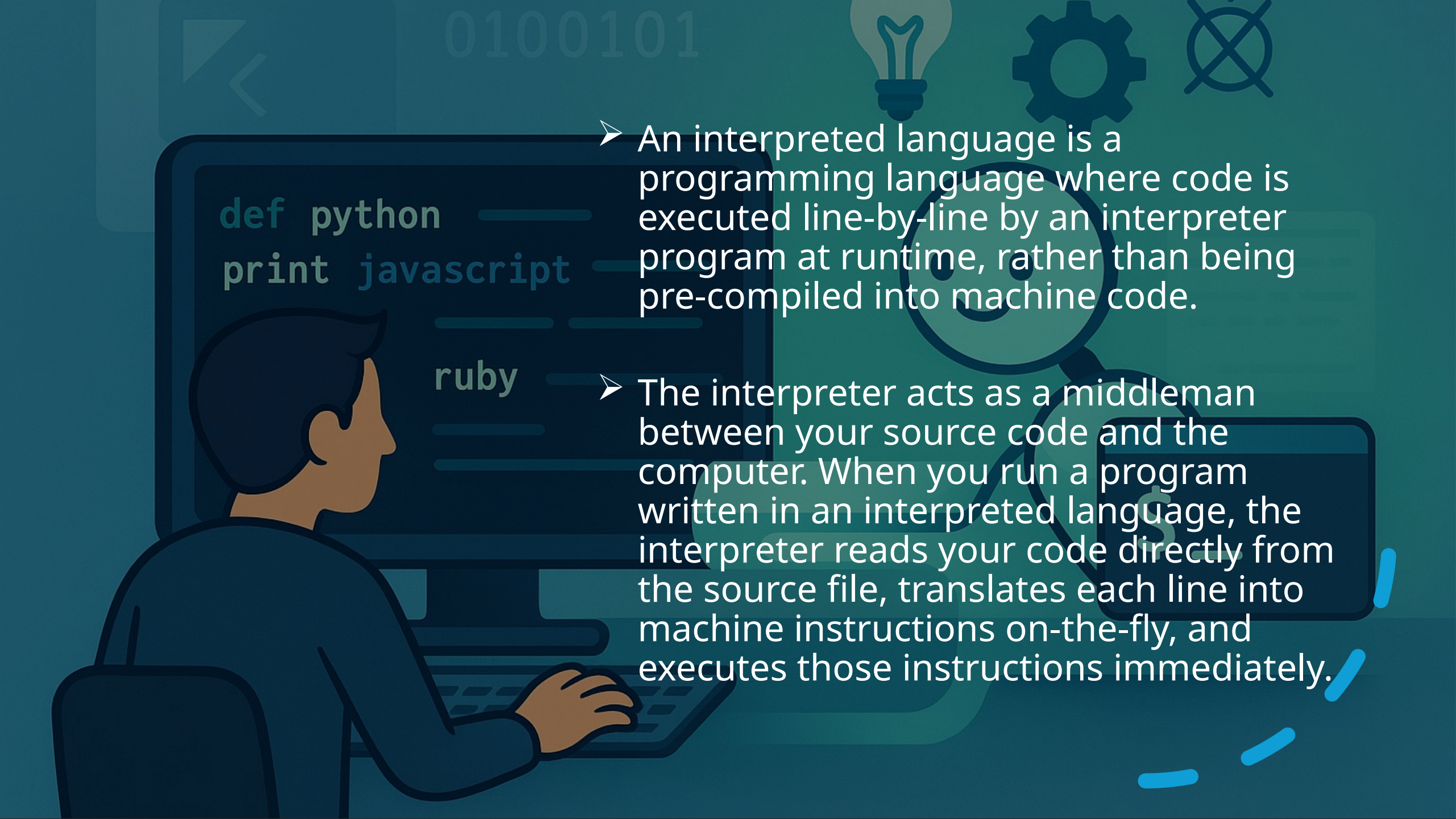


Mobile and Embedded Computing

Lecture 2. Languages & Flutter Intro



- A compiled language is a programming language that must be translated into machine code before it can run.
- Uses a compiler to convert the code into an executable file that the computer can directly run. This translation step happens once, before you run the program.



The background is a dark teal color. At the top, there are faint icons: a lightbulb, a gear, and a crossed-out circle. In the top left, there's a faint binary code '0100101' and a back arrow. A person is shown from the back, sitting at a desk and typing on a laptop. A large magnifying glass is positioned over the text on the right. The text on the left is in a light blue font, and the text on the right is in a white font.

```
def python  
print javascript
```

```
ruby
```

- An interpreted language is a programming language where code is executed line-by-line by an interpreter program at runtime, rather than being pre-compiled into machine code.
- The interpreter acts as a middleman between your source code and the computer. When you run a program written in an interpreted language, the interpreter reads your code directly from the source file, translates each line into machine instructions on-the-fly, and executes those instructions immediately.

Differences – Developer Experience – Compiled languages

Slow to develop (edit, compile, link and run. The compile/link steps could take serious time)

You need to "rebuild" the program every time you need to make a change

Compiled languages require longer development cycles but catch errors earlier in the development process

Early error detection during compilation prevents runtime failures

Better for large, complex applications where catching bugs early saves time

Differences – Developer Experience – Interpreted languages

Interpreted languages accelerate development cycles through immediate code execution and dynamic testing capabilities. Developers can modify and test code without compilation delays

Since there is no compilation step, changes to the code can be immediately executed and tested, allowing for quick iterations and experimentation

Interpreted languages, on the other hand, offer greater flexibility and a shorter development feedback loop, which makes them well-suited for scripting, prototyping, and rapid application development

Better for rapid prototyping, web development, and scenarios requiring frequent updates



Differences – Runtime Execution Speed

Compiled languages tend to be about 100 times faster than interpreted languages

Compiled languages generally outperform interpreted languages in execution speed benchmarks. Machine code execution eliminates interpretation overhead, resulting in faster program execution

Performance differences matter most in computationally intensive applications like scientific computing, real-time systems, etc

Differences – Security

01

Compiled code is difficult to reverse engineer or modify

02

Since the source code is often distributed alongside the interpreter, it may be easier to access and reverse engineer proprietary algorithms

03

Compiled languages catch more errors at compile-time, preventing security vulnerabilities from reaching production

04

Interpreted languages offer more runtime flexibility but this can be exploited maliciously



Kotlin
Dart
Swift

The illustration features a person with dark hair, wearing a green shirt, sitting at a desk and typing on a keyboard. Behind them is a large green shield with a white checkmark. To the left of the shield is a red circle with a diagonal line through it and the word 'NULL' in red. Above the shield is a green padlock icon and a green circle with a white checkmark. To the right of the shield is a green arrow pointing towards a document icon with horizontal lines. The background is a light green gradient with some faint horizontal lines.

Null safe
languages



Null safe languages

Null safety refers to a programming language's ability to prevent null pointer exceptions (NPEs) at compile time rather than runtime. The type system distinguishes between types that can hold null values and those that cannot, making null-related errors largely impossible when the code compiles successfully.

Java Not null safe

```
1 record User(String name, String email) { 11 usages
2 }
3
4 class UserService {
5     public void processUser(User user) { 3 usages
6         // This can throw NullPointerException at runtime
7         System.out.println("Processing: " + user.name().toUpperCase());
8
9         // Even checking for null doesn't guarantee safety
10        if (user.email() != null) {
11            System.out.println("Email: " + user.email().toLowerCase());
12        }
13    }
14
15    static void main() {
16        User user1 = new User(name: "John", email: "john@example.com");
17        User user2 = new User(name: null, email: null); // Compiles fine!
18        User user3 = null; // Also compiles fine!
19
20        UserService service = new UserService();
21        service.processUser(user1); // Works fine
22        service.processUser(user2); // Runtime NPE on getName().toUpperCase()
23        service.processUser(user3); // Runtime NPE on user.getName() Passing 'null'
24    }
25 }
```

Dart – Null safe

```
1 class User {  
2   final String name;    // Non-nullable - cannot be null  
3   final String? email;  // Nullable - can be null (note the ?)  
4  
5   User({required this.name, this.email});  
6 }  
7  
8 class UserService {  
9   void processUser(User user) {  
10    // This is guaranteed safe - name cannot be null  
11    print('Processing: ${user.name.toUpperCase()}');  
12  
13    // Compiler forces null checking for nullable types  
14    if (user.email != null) {  
15      // Smart cast: email is promoted to non-null String inside this block  
16      print('Email: ${user.email!.toLowerCase()}');  
17    }  
18  
19    // Alternative null-aware operators  
20    print('Email length: ${user.email?.length ?? 0}');  
21  }  
22 }  
23
```

```
24 >> void main() {  
25   var user1 = User(name: 'John', email: 'john@example.com');  
26   var user2 = User(name: 'Jane'); // email is null, but explicitly nullable  
27  
28  
29   // This won't compile - name is required and non-nullable  
30   // var user3 = User(name: null); // Compilation error!  
31  
32   // This won't compile - User itself is non-nullable  
33   // User user4 = null; // Compilation error!  
34  
35   var service = UserService();  
36   service.processUser(user1); // Safe  
37   service.processUser(user2); // Safe - null handling is explicit  
38  
39   // print(user2.email!.toLowerCase()); // Unsafe - will throw if email is null  
40   // print(user2.email?.toLowerCase() ?? 'No email provided'); // Safe alternative  
41 }
```

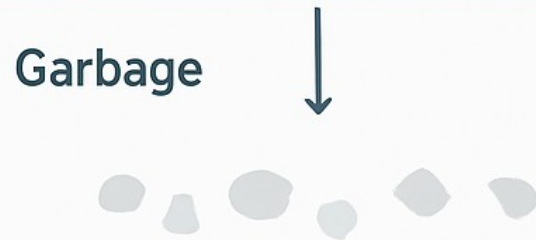
Dart

- client-optimized programming language developed by Google
- released in 2011
- Strong typed
- Null safe
- Object Oriented
- Built in support for Async programming
- Garbage collected
- Can hot reload

Dart Garbage Collector

- Two generation approach (Young generation and Old generation)
- Objects that survive multiple young generation collections are **promoted** to the old generation.
- The Dart VM implements **concurrent marking** for old generation collections. The marking phase runs concurrently with the application (mutator), reducing pause times. The collector uses **write barriers** to track modifications made by the application during concurrent marking, ensuring collection correctness.
- **Incremental Sweeping** The sweep phase can be performed incrementally, spreading the work across multiple small time slices rather than completing it all at once. This further reduces pause times by avoiding long stop-the-world phases.

Young Generation (New Space)



Objects surviving
multiple scavenges
get promoted



Old Generation (Mark-Sweep-Compact)

Mark Phase



Sweep Phase



Compact Phase



Large Objects



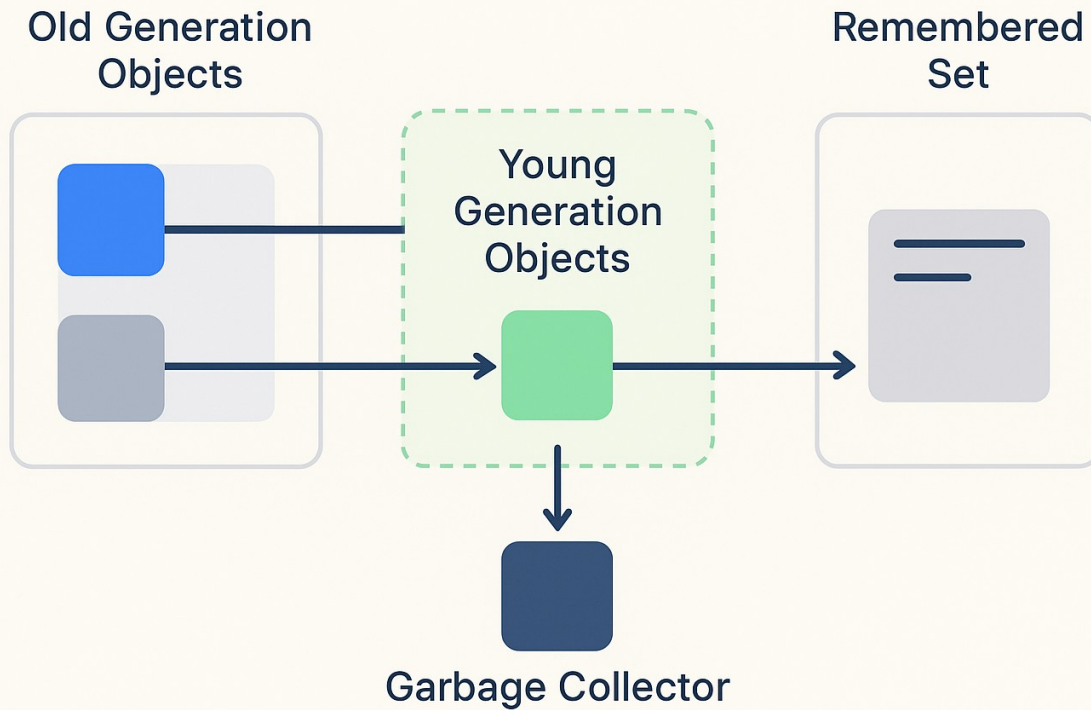
Write Barriers
+ Remembered
Sets



Concurrent Marking



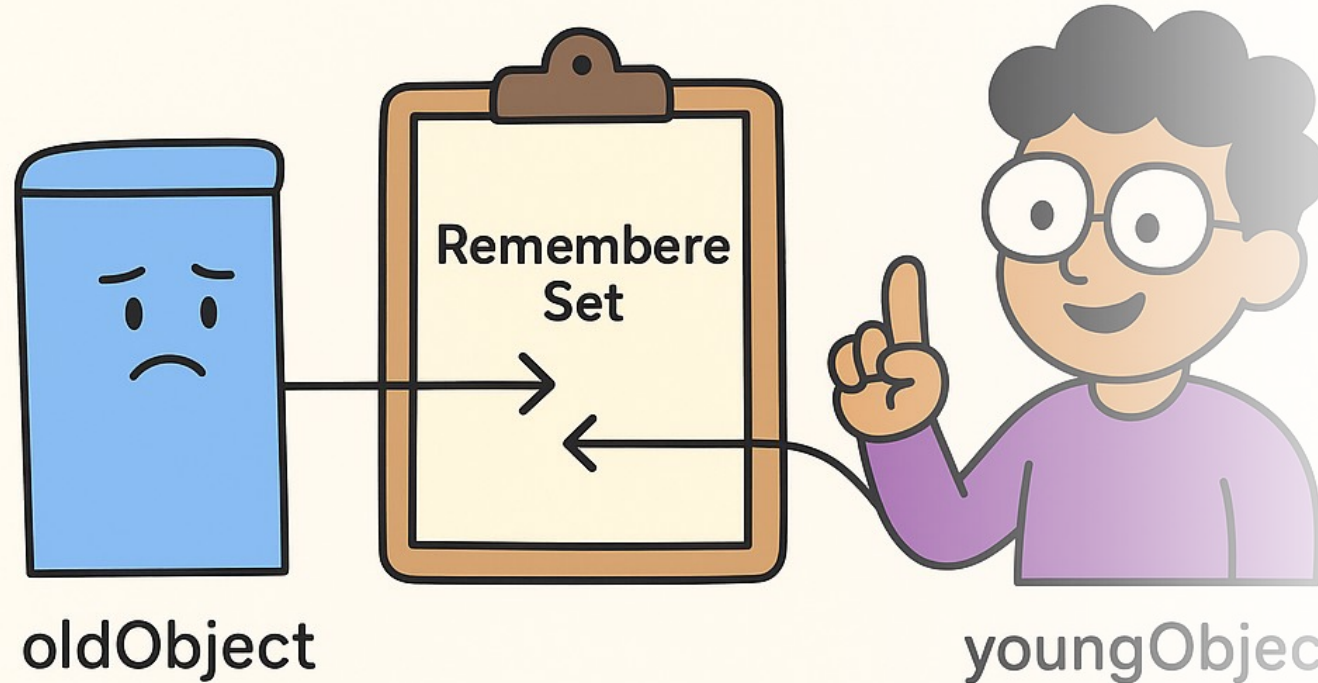
Incremental Sweeping



•In a generational garbage collector, the young generation is collected independently and much more frequently than the old generation. However, this creates a fundamental problem: **what if an old generation object holds the only reference to a young generation object?**

•A **remembered set** is a data structure that tracks references from the old generation to the young generation. Think of it as a "reminder list" that tells the young generation collector: "Don't forget to check these old objects when determining what's reachable in the young generation."

Remembered Set



- ✓ Contains pointers from old → young
- ✓ Used as extra roots during GC
- ✓ Ensures young objects are preserved

The remembered set typically contains:

- Pointers to old generation objects that reference young generation objects
- The specific memory locations (slots) within old objects that contain young generation references
- When performing a young generation collection, the garbage collector treats the remembered set as additional "roots" for reachability analysis. It scans all objects referenced in the remembered set to find young generation objects that should be preserved.

Write Barrier



- ✓ Is the target in old generation?
- ✓ Is the new in young generation?
- ✓ Add to remembered set

A write barrier is a small piece of code that executes whenever the application modifies an object reference. It acts as a "hook" that allows the garbage collector to monitor and respond to changes in the object graph.

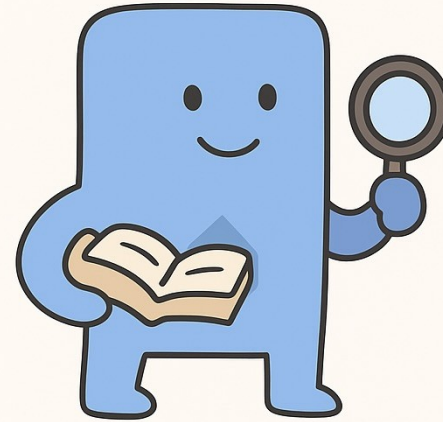
When the application executes code like `oldObject.field = youngObject`, the write barrier triggers and performs these steps:

1. Is the target object (`oldObject`) in the old generation?
2. Is the new reference (`youngObject`) in the young generation?

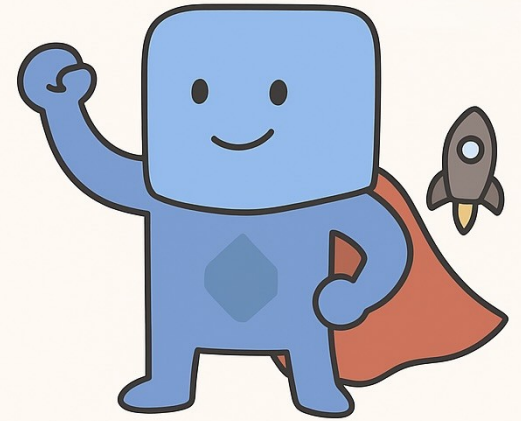
=> If both conditions are true, add information about this reference to the remembered set

Dart VM

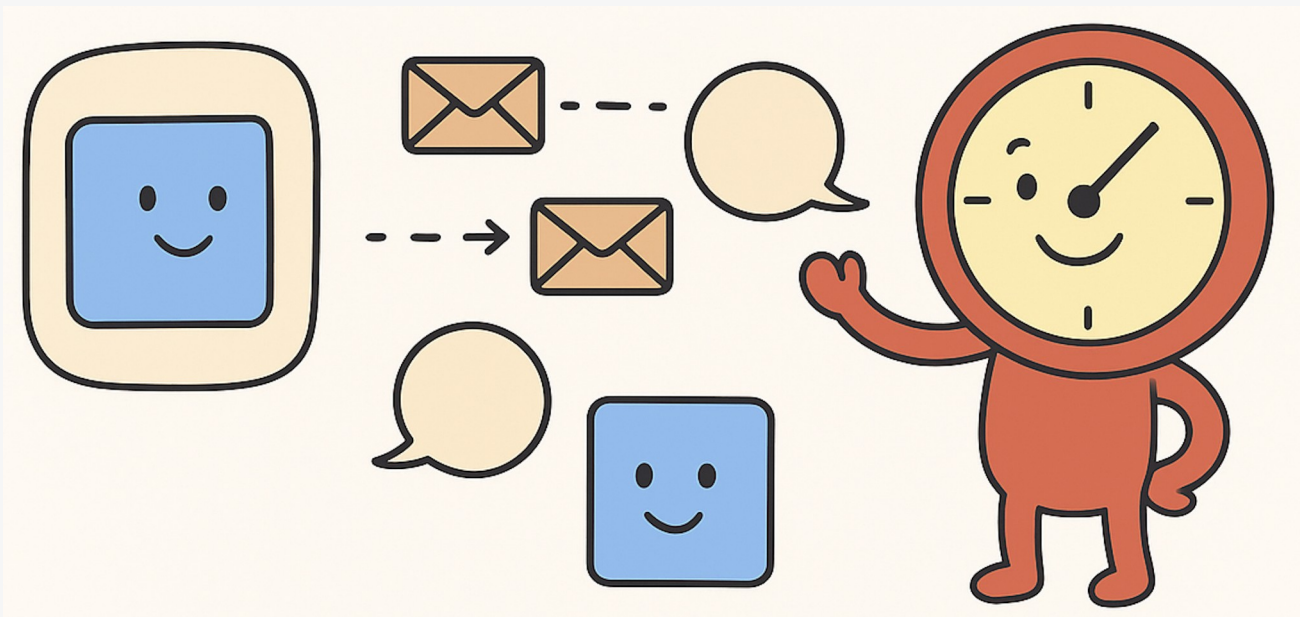
- During development operates in JIT mode (Dart source code is initially parsed and executed through an interpreter)
- In production runs in AOT mode (compiled ahead-of-time to native machine code, eliminating the VM overhead entirely. AOT compilation produces standalone executables or libraries that start faster and consume less memory than JIT-compiled applications)



Development: JIT
(interpreter + compiler)



Production: AOT
(native machine code)



Dart

concurrency

- Isolates (independent workers that communicate through message passing, each Isolate has its own memory heap and event loop)
- Async

Flutter





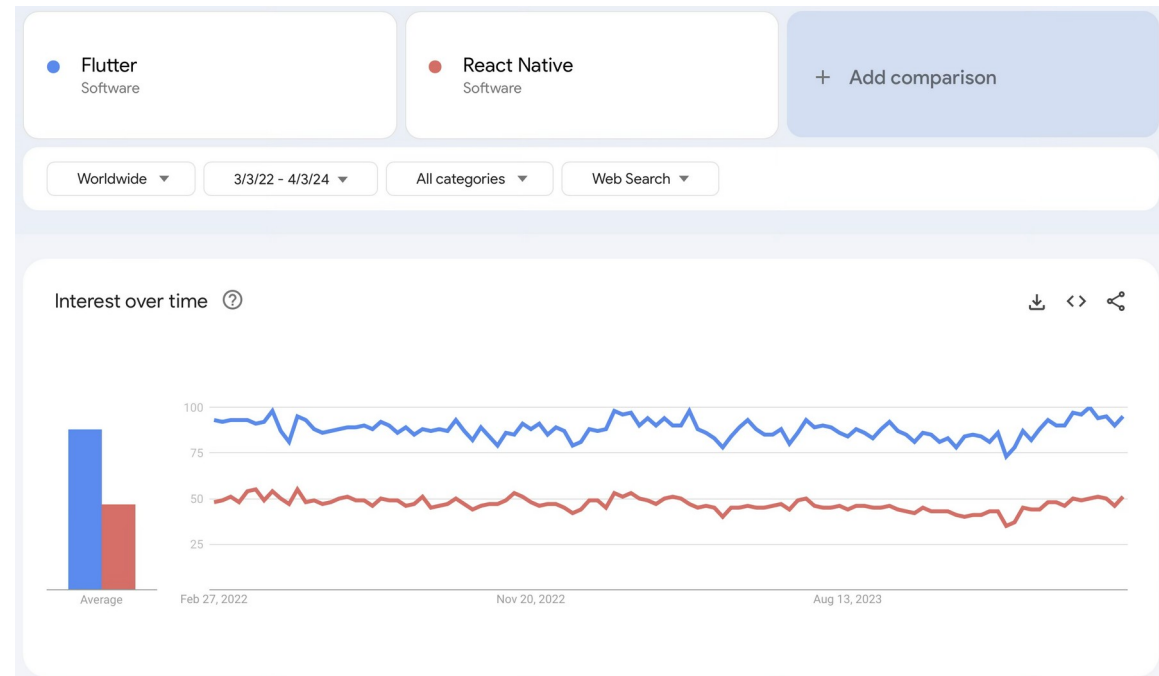
Surface™



BMW

Scaling customer-centric product development at BMW Group with Flutter

[Read the story →](#)





250k+ installs

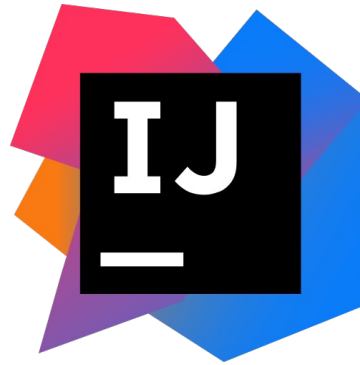
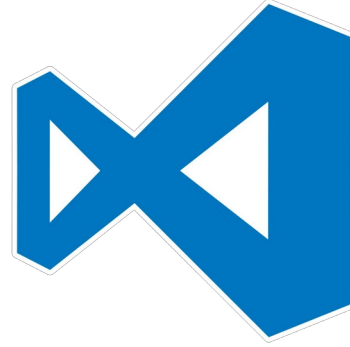
Flutter + Spring Boot, Firebase

<https://scanneralimente.ro>

+45 other apps

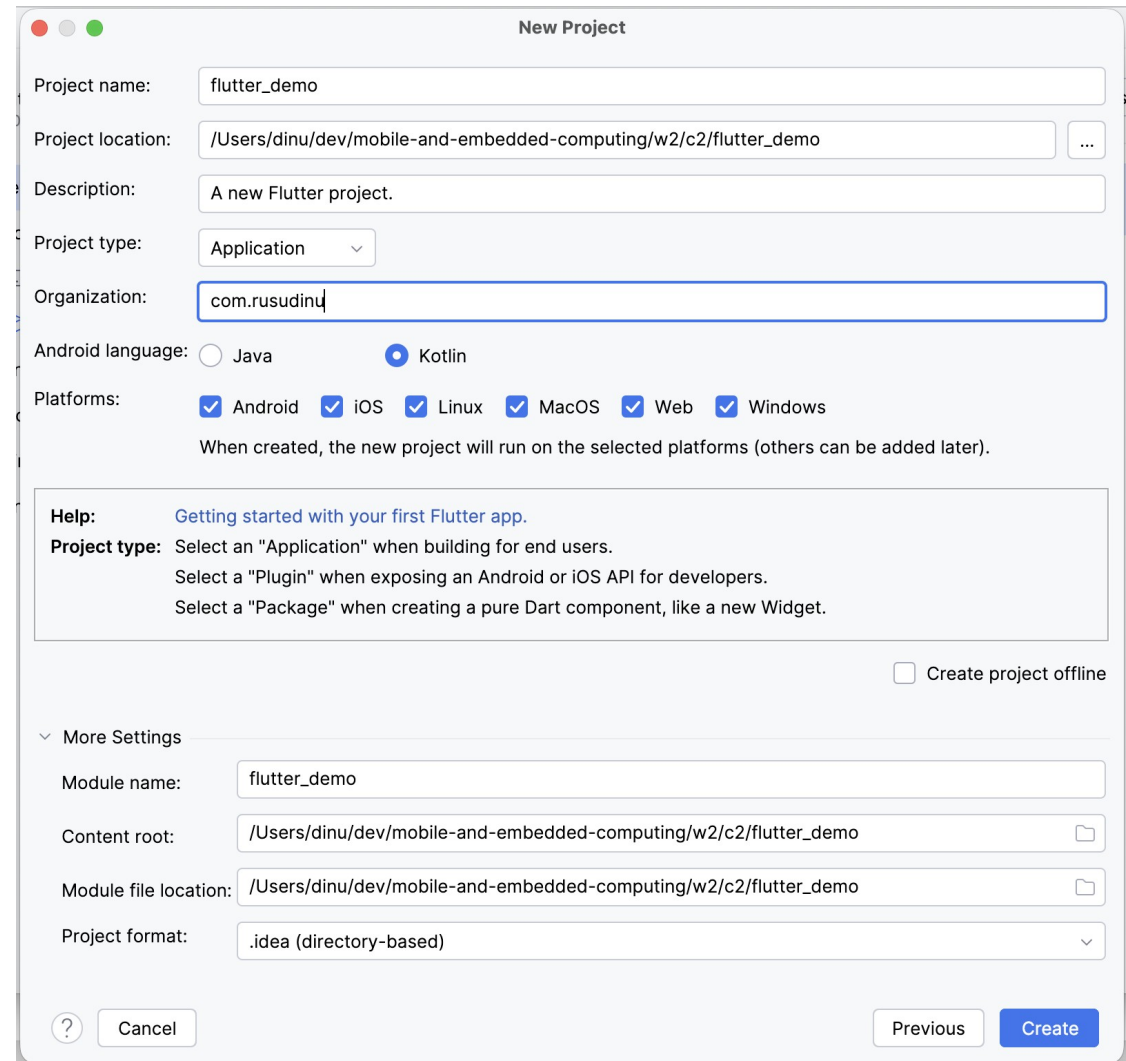
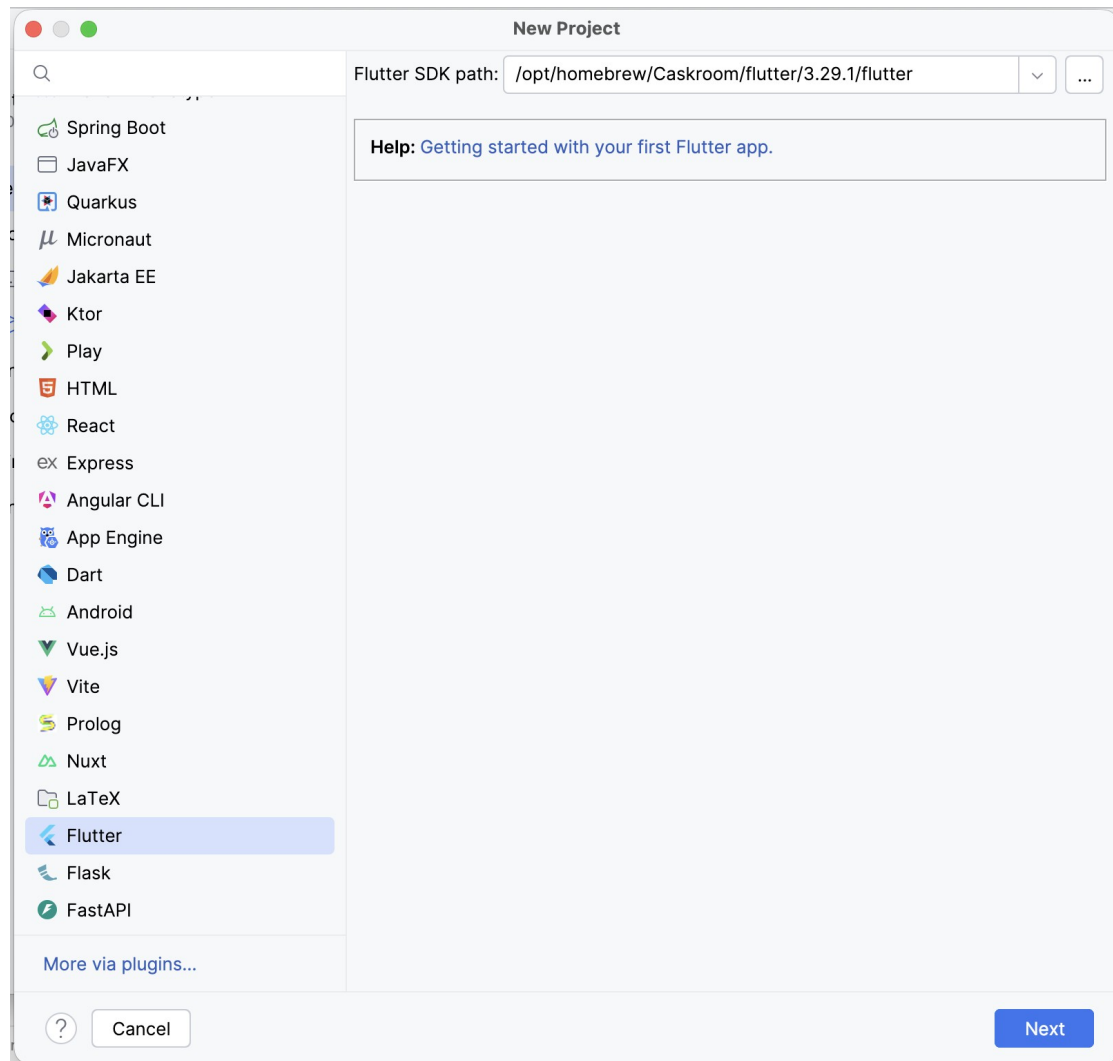


- Google's open-source UI toolkit for building natively compiled applications across multiple platforms from a single codebase
- Uses the Dart programming language and provides a rich set of pre-designed widgets that follow platform-specific design guidelines
- Excels at creating cross-platform mobile applications (iOS and Android) with almost native performance
- Well-suited for apps requiring custom UI designs, complex animations, and rapid prototyping
- Supports web and desktop development
- Hot reload
- Widget-based architecture
- Not be ideal for applications requiring extensive platform-specific integrations, heavy reliance on native device APIs, or projects where app size is critical (Flutter apps tend to be larger than native



Flutter and your development environment:

- Flutter SDK (<https://docs.flutter.dev/install>) (click on Install manually)
- Android Studio / Xcode
- IntelliJ or VS Code to write the code (with the Flutter and Dart plugins)



```
main.dart x
1 import 'package:flutter/material.dart';
2
3 void main() {
```

yApp();

nds StatelessWidget {
uper.key});

is the root of your application.

uildContext context) {

ialApp(
utter Demo',
meData(
is the theme of your application.

HIS: Try running your application with "flutter run". You'll see
pplication has a purple toolbar. Then, without quitting the app,
hanging the seedColor in the colorScheme below to Colors.green
hen invoke "hot reload" (save your changes or press the "hot
d" button in a Flutter-supported IDE, or press "r" if you used
ommand line to start the app).

e that the counter didn't reset back to zero; the application
is not lost during the reload. To reset the state, use hot
rt instead.

idency constraints.

macOS (desktop)

Current File



Chrome (web)

macOS (desktop)

rusu's iPhone (wireless) (mobile)

Open iOS Simulator

Open Android Emulator: Medium Phone API 35

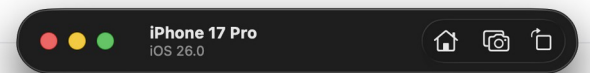
Open Android Emulator: Pixel 9 Pro

Open Android Emulator: Pixel 9 Pro XL

Open Android Emulator: Pixel 9a

Open Android Emulator: Pixel Tablet

Restart Flutter Daemon



11:17

Flutter Demo Home Page

You have pushed the button this many times:

0



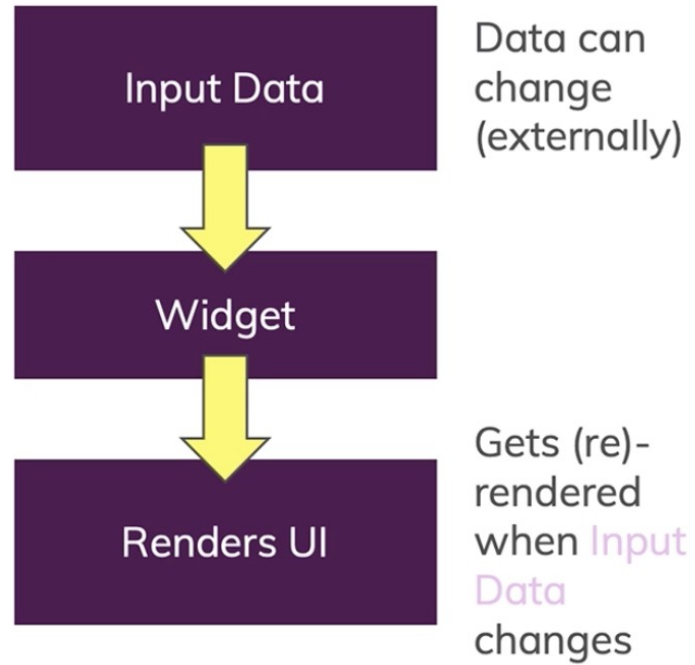
Phone 17 Pro in debug mode...

14.5s

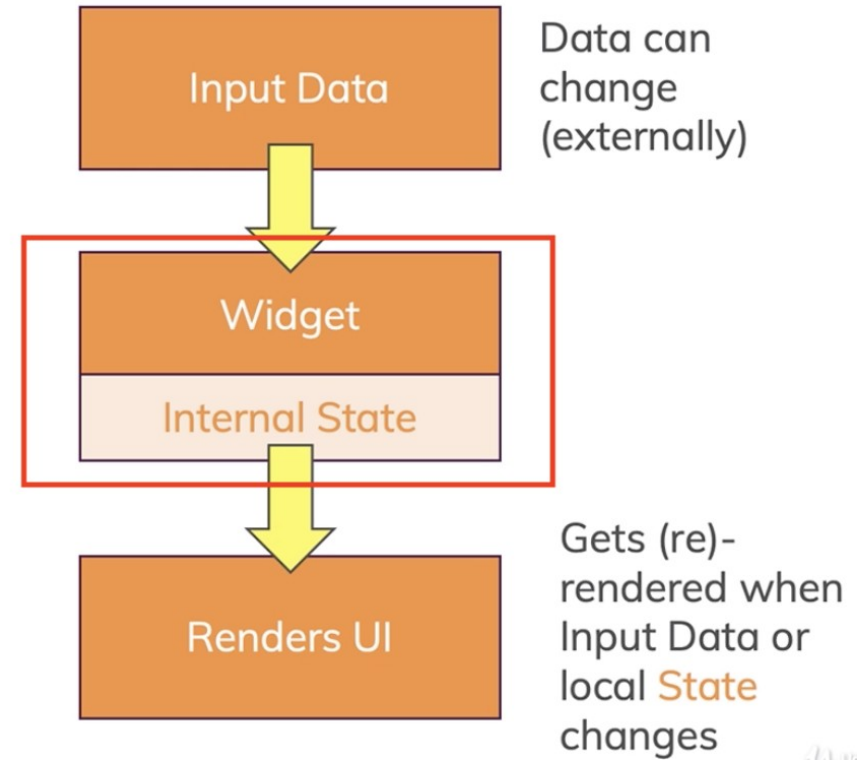
s://127.0.0.1:62013/TwUNq9UNa4A=/ws
ne 17 Pro...

g for Dart.

Stateless



Stateful



Stateless vs Stateful

Flutter Widget Lifecycle

